



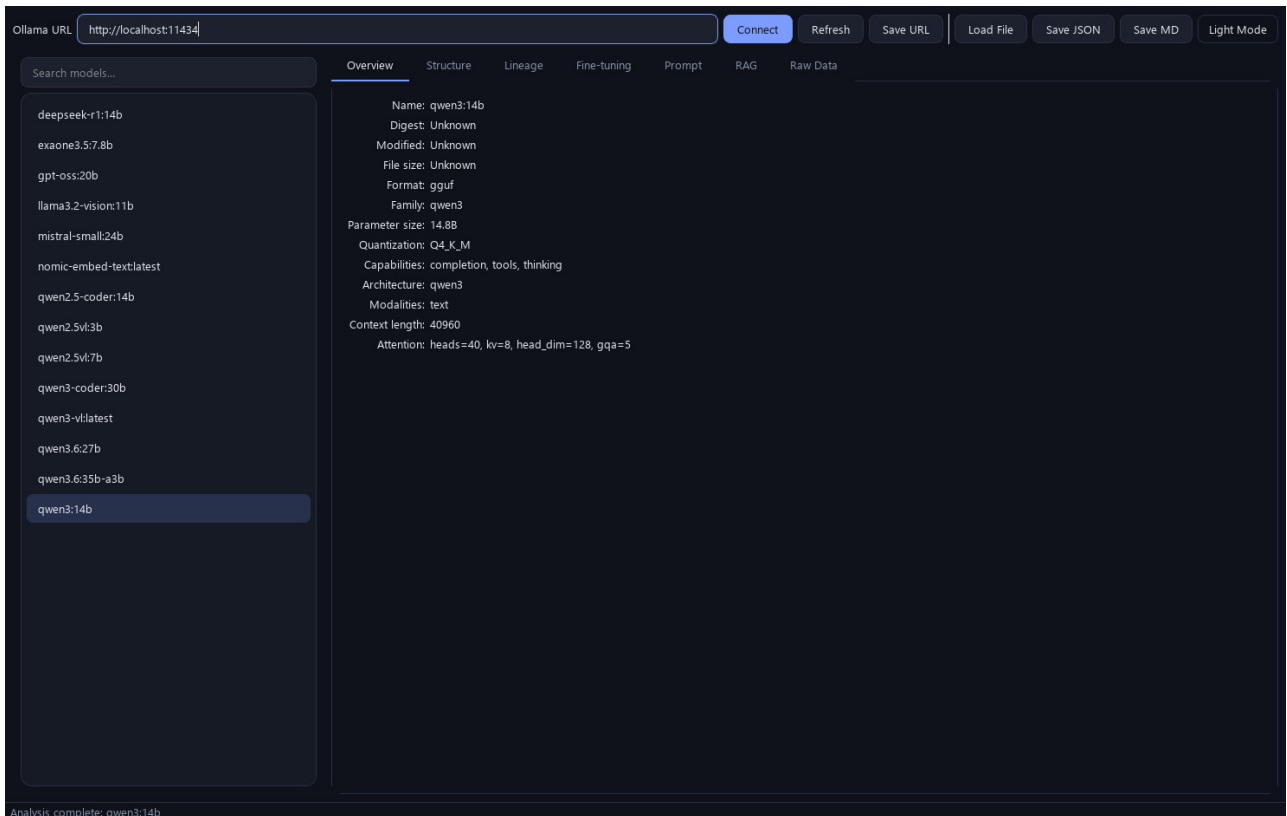
Ollama Model & RAG Inspector

User Manual · Version 1.0.0

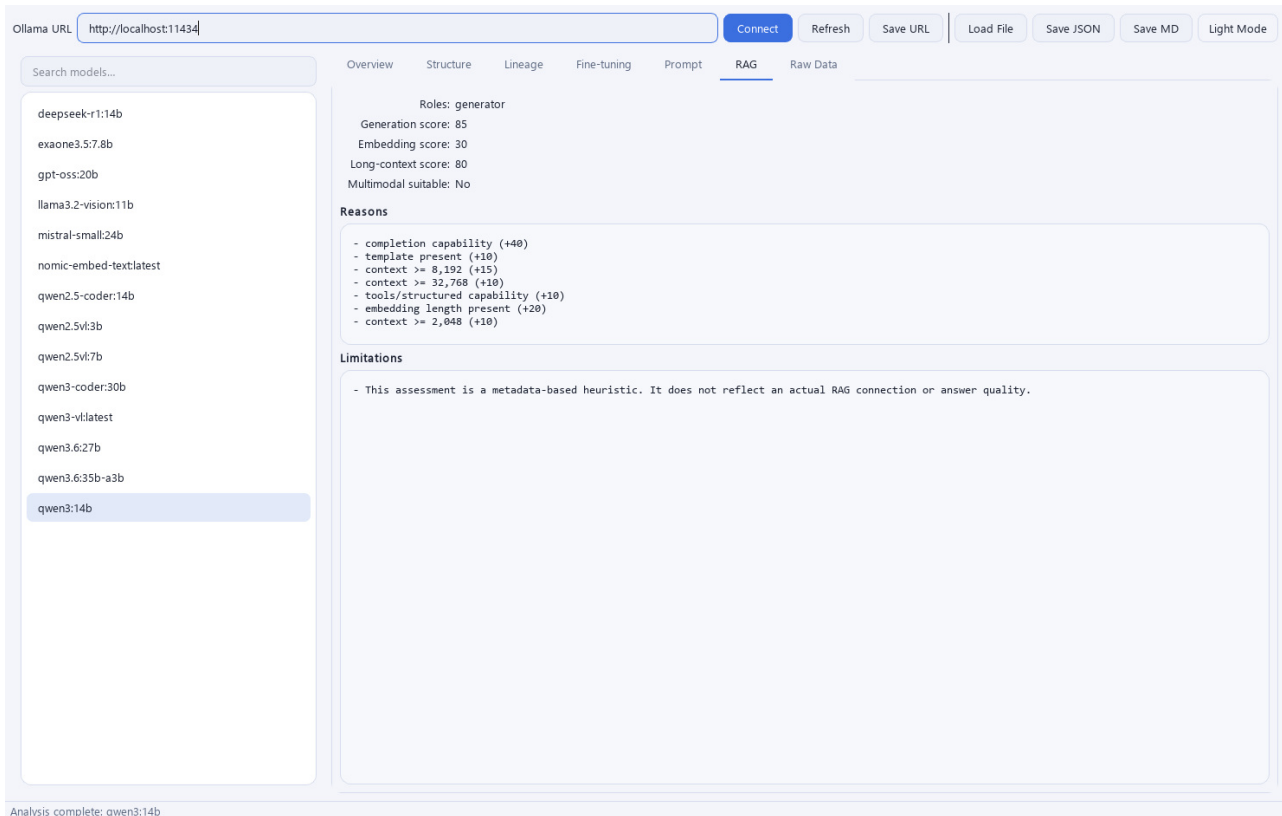
A read-only desktop application (PySide6) that inspects models registered on an **Ollama** server, and external **GGUF/Modelfile** files on disk. It reports basic info, architecture metadata, Modelfile configuration, model lineage, fine-tuning traces, and RAG role-suitability. Every conclusion is **evidence-based, never guessed**.

Screenshots

Connected to a live Ollama server — the Overview tab (dark theme):



The RAG role-suitability tab (light theme):



1. Key principles

- **Read-only & safe.** No pull, create, delete, copy, Modelfile edit, or inference. Only `/api/tags`, `/api/show`, and optionally `/api/ps` are called.
- **Generic.** No model name is hardcoded; unknown architectures still show raw metadata and any discoverable structural values.
- **Evidence-based.** Every fact carries a status; anything unverifiable stays *Unknown* instead of being guessed.
- **Partial success.** If one analyzer fails, the rest is still shown with a warning.

2. Requirements

- Python 3.11 or newer.
- An Ollama server to inspect (default `http://localhost:11434`), and/or local `.gguf` / `Modelfile` files.

3. Installation

Windows (batch helpers):

```
setup.bat  :: create .venv and install the app + dev tools
run.bat    :: launch the GUI (auto-runs setup.bat if needed)
test.bat   :: run tests + lint headless
```

Manual (any OS):

```
python -m venv .venv
pip install -e ".[dev]"
python -m ollama_inspector.main
```

4. Getting started

1. Confirm or edit the **Ollama URL** and click **Connect**.
2. Pick a model from the left list. The **search box** filters by name.
3. Browse the detail tabs (below).

4. Save a report with **Save JSON** or **Save MD**.

The UI never freezes: all network and analysis work runs on background threads, and a request-ID guard discards stale results when you switch models quickly.

5. The tabs

Tab	Shows
Overview	Name, digest, size, format, family, parameter size, quantization, capabilities, architecture, modalities, context length, attention.
Structure	Canonical architecture values, the raw metadata tree, and supporting evidence.
Lineage	Current / base / parent model, adapters, and the detected derivation type with evidence.
Fine-tuning	Fine-tuned yes/no/unknown, method, adapters, training-metadata presence, and the list of facts left Unknown.
Prompt	Modelfile, SYSTEM, TEMPLATE, PARAMETERS, LICENSE.
RAG	Roles and heuristic scores (generation, embedding, long-context), multimodal suitability, reasons, and limitations.
Raw Data	The full <code>/api/show</code> payload, <code>model_info</code> , <code>details</code> , and capabilities.

6. Evidence statuses

Status	Meaning
Confirmed	Read directly from the API or Modelfile.
Derived	Computed from confirmed values.
Detected	Inferred from a rule or pattern.
Declared	From external configuration or user input.
Unknown	Could not be determined.
Conflict	Data sources disagree.

7. Inspecting external models

Click **Load File** to inspect a model stored outside Ollama:

- A local **.gguf** file — its GGUF metadata header is parsed in pure Python (*no weights are loaded*) and run through the same analyzers.
- A **Modelfile**, or a **folder** containing either.

Because a raw GGUF has no capability/Modelfile information, some fields will be limited — the app says so rather than guessing.

8. Themes

Use the top-right **Dark Mode / Light Mode** button to switch themes at any time. The choice is remembered between sessions. The layout is tuned for Full HD displays.

9. Exporting reports

- **JSON** — the full result including the raw response (UTF-8, indent 2).
- **Markdown** — Summary, Architecture, Capabilities, Lineage, Fine-tuning, Prompt Configuration, RAG Suitability, Warnings, Evidence.

10. Building a standalone executable (Windows)

```
build_exe.bat      :: generates the icon and builds dist\OllamaInspector.exe
```

Two PyInstaller specs are provided: `OllamaInspector.spec` (single file) and `OllamaInspector-onedir.spec` (a folder where the Qt libraries are separate files). The bundled executable includes Qt, used under the **LGPL-3.0**; distributing it carries LGPL obligations — see `THIRD_PARTY_LICENSES.md`.

11. Troubleshooting

Symptom	What to do
Connection failed	Check the URL and that <code>ollama serve</code> is running. The app stays open; fix the URL and click Connect again.
Analysis failed for one model	The model list is kept; the error is shown and you can select a different model.
"Not a GGUF file"	The file is not a valid GGUF (or is version 1); try a Modelfile or a GGUF v2/v3 file.

12. Scope & limitations

The primary data source is the Ollama HTTP API. For external files, only GGUF *metadata* (header + key/values) is read. The app does **not** read GGUF tensors/weights, access Ollama manifests/blobs, run inference, or evaluate answer quality. RAG scores are metadata-based heuristics and do not reflect an actual RAG connection.

13. License

This project is released under the **MIT License**. Bundled/third-party components keep their own licenses — notably PySide6/Qt (LGPL-3.0). See `LICENSE` and `THIRD_PARTY_LICENSES.md`.